

Code-Layout, Readability and Reusability

Date	10 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	2 Marks

Project Folder Structure

Folder / File	Purpose
data/	Raw and processed applicant datasets (CSV)
notebooks/	Jupyter notebooks for EDA, preprocessing, and model experimentation
src/preprocessing.py	Reusable functions for cleaning, encoding, and scaling input data
src/train_model.py	Trains Logistic Regression, Random Forest, XGBoost, and Decision Tree models
src/predict.py	Loads the saved model and generates predictions for new applicants
models/best_model.pkl	Serialized best-performing trained model
app.py	Flask application entry point and routes
templates/	HTML templates for the input form and result page
static/	CSS and static assets for the web app
requirements.txt	List of Python package dependencies

Code Quality Practices Followed

📁 Layout

- Consistent indentation and PEP-8 style formatting
- Logical separation of preprocessing, training, and app code
- Configuration values (paths, thresholds) kept at the top of scripts

📖 Readability

- Descriptive function and variable names (e.g., encode_employment_type)
- Docstrings and inline comments explaining non-obvious logic
- Meaningful commit messages describing each change

♻️ Reusability

- Preprocessing pipeline reused identically in training and inference
- Model training function accepts any classifier as a parameter
- Flask routes call shared helper functions rather than duplicating logic

🔧 Maintainability

- Modular files make it easy to swap or retrain the model
- requirements.txt keeps dependencies reproducible
- Clear separation between data, model artifacts, and app code